

VANITY FAIR



WHY YOUR MODULES DESERVE BETTER THAN GITHUB.COM

Because your import path is your first impression.



presented by

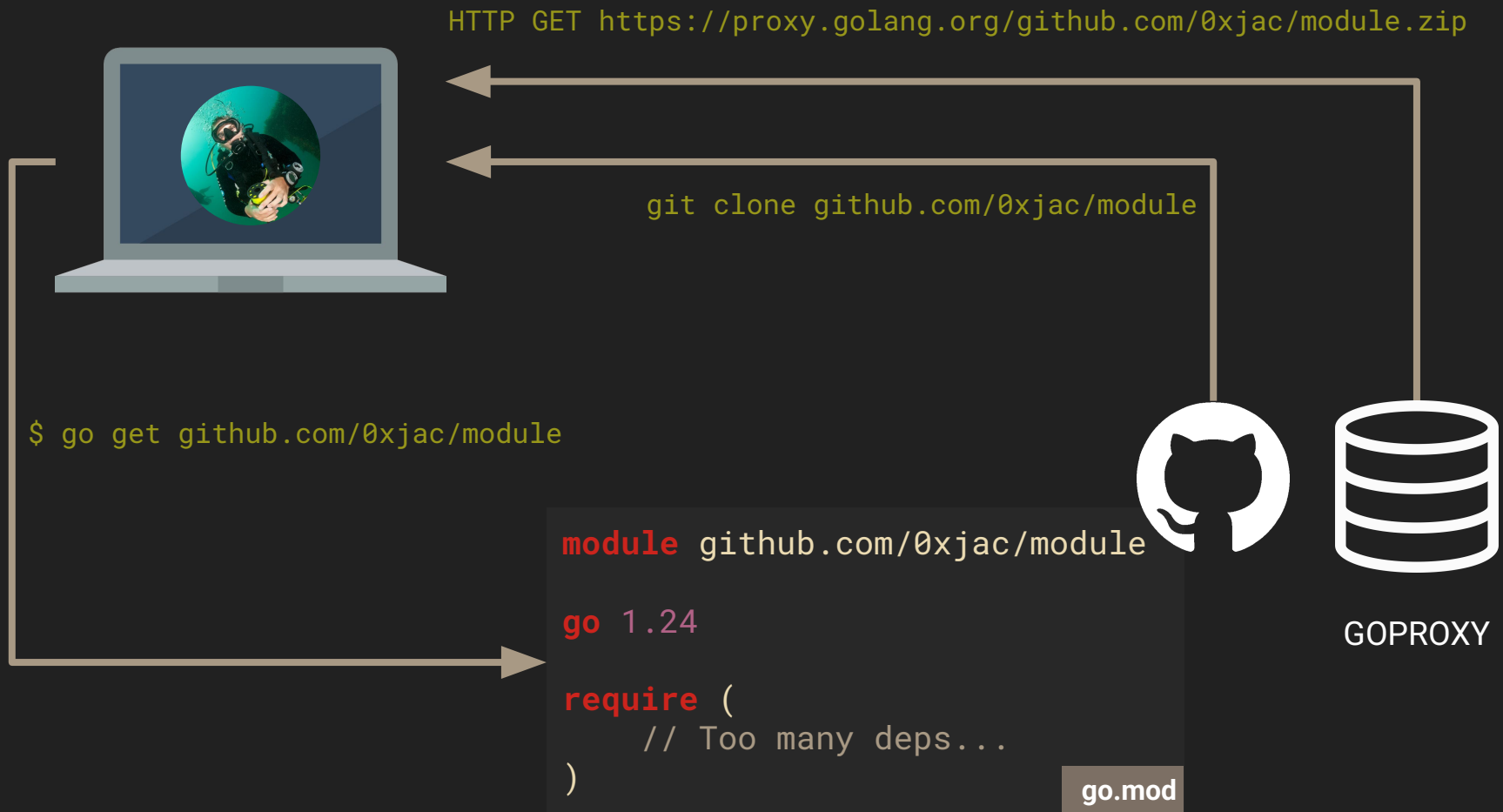
Jacques Dafflon | 0xjac

0xjac.com

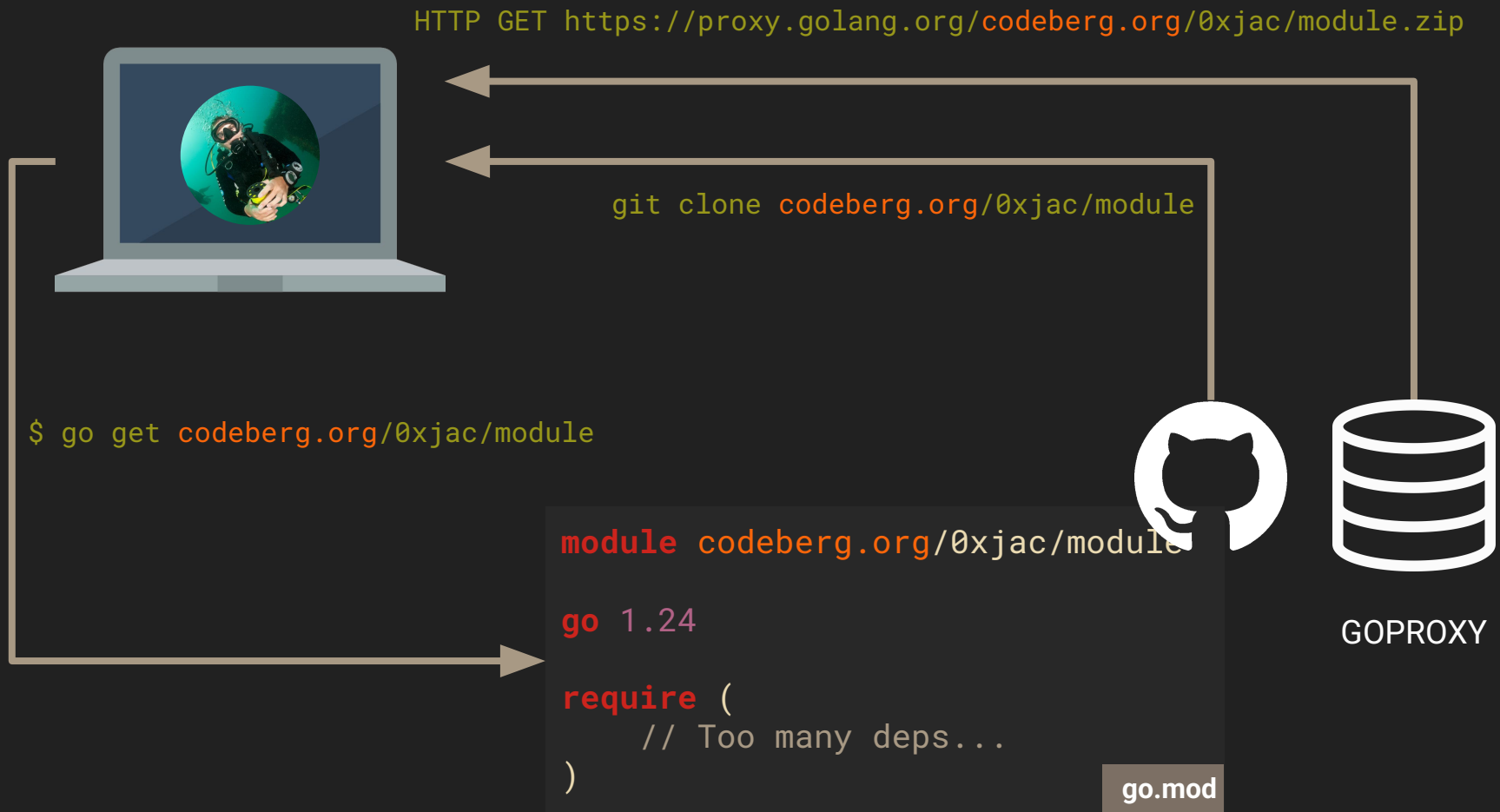
Go Bärner Meetup

September 3rd 2025

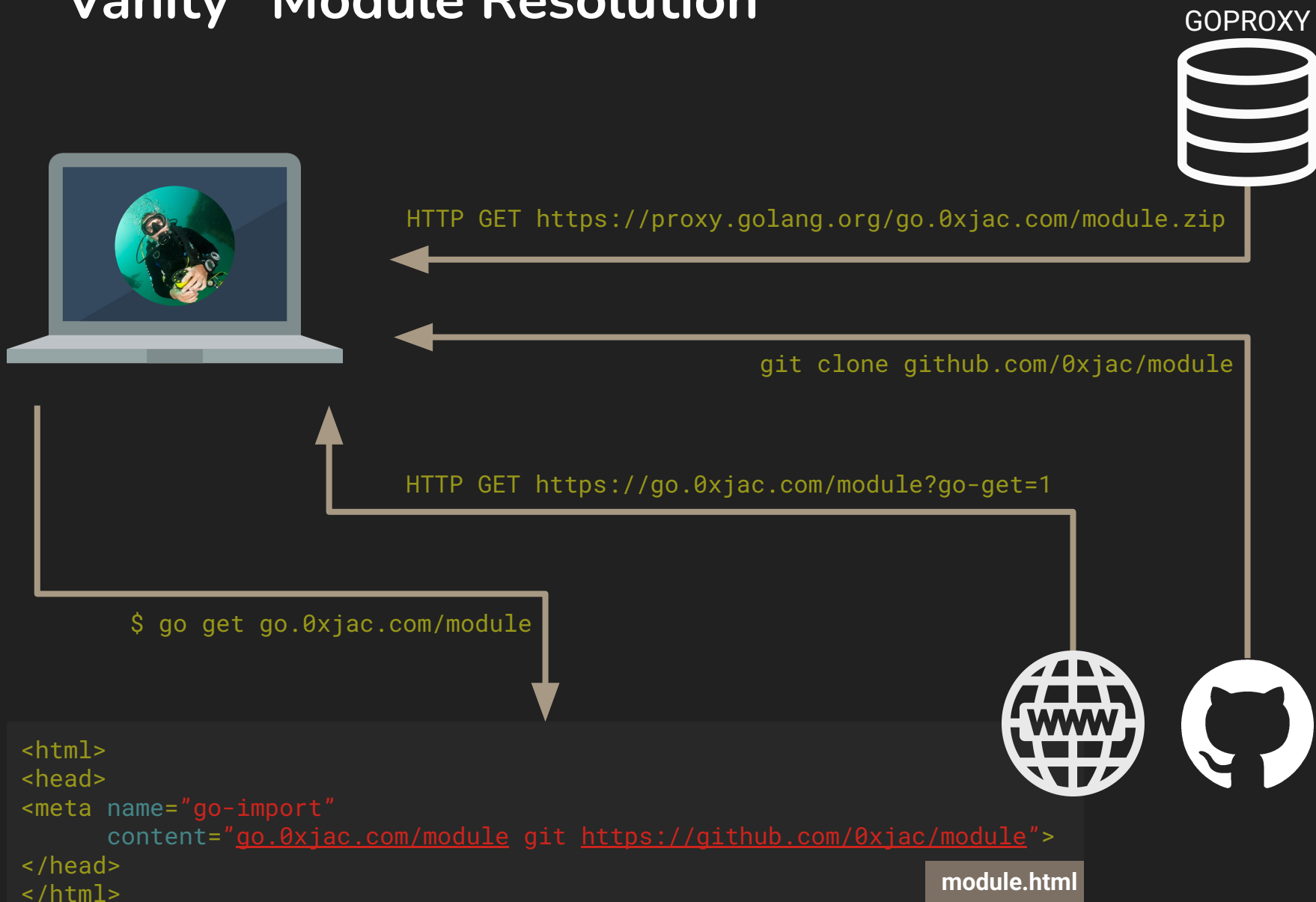
Module Resolution



Module Resolution



“Vanity” Module Resolution



Go-Import Meta Tag

```
<meta name="go-import" content="root-path vcs repo-url">
```

Canonical path
of your module
(as defined in `go.mod`)

One of:

`bzr`
`fossil`
`git`
`hg`
`svn`

Root of your VCS
repo holding the
code

```
<meta name="go-import"  
content="go.0xjac.com/module git https://github.com/0xjac/module">
```

Follows redirections and ignores status codes!

Reference: go.dev/ref/mod#vcs-find

The VCS Qualifier: You may have been using “vanity” imports without knowing it!

If the module path has a VCS qualifier (one of .bzd, .fossil, .git, .hg, .svn) at the end of a path component, the go command will use everything up to that path qualifier as the repository URL

github.com/0xjac/module

OOPS!

```
$ go get github.com/0xjac/module.git
```

invalid version control suffix in github.com/ path

Special case detected as `git`, but github.com also includes:

```
<meta name="go-import" content="github.com/0xjac/module git  
https://github.com/0xjac/module.git">
```

Gitlab.com & Codeberg.org are using vanity imports!

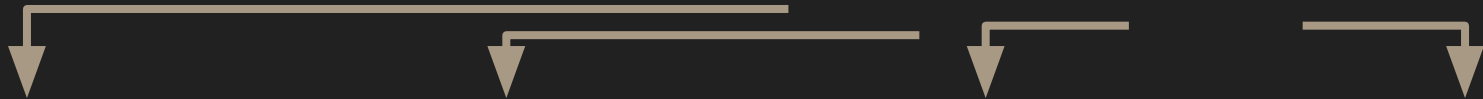
Go-source Meta Tag

Not part of the mod specs!

Allows the doc on pkg.go.dev to link to the source code.

See: pkg.go.dev/about#source-links

```
<meta name="go-source" content="prefix home directory file">
```



import path
(repository root)

URL of the repository's
home page

URL template for a
page listing the files in
the package with
{dir} or {/dir}.

URL template for a link
to a line in a source file
with {file} and
{line}.

```
<meta name="go-source" content="
  go.0xjac.com/module
  https://github.com/0xjac/module
  https://github.com/0xjac/module/tree/master/{/dir}
  https://github.com/0xjac/module/tree/master/{/dir}/{file}#{line}"
>
```

>

Examples In The Wild

golang.org/x/...

```
<meta name="go-import" content="golang.org/x/text" git https://go.googlesource.com/text>  
<meta http-equiv="refresh" content="0; url=https://pkg.go.dev/golang.org/x/text">
```

google.golang.org/protobuf

```
<meta name="go-import"  
      content="google.golang.org/protobuf" git https://go.googlesource.com/protobuf>  
<meta name="go-source" content="google.golang.org/protobuf  
https://github.com/protocolbuffers/protobuf-go  
https://github.com/protocolbuffers/protobuf-go/tree/master{/dir}  
https://github.com/protocolbuffers/protobuf-go/tree/master{/dir}/{file}#L{line}">  
<meta http-equiv="refresh" content="0; url=https://pkg.go.dev/google.golang.org/protobuf">
```

gopkg.in/yaml.v1

k8s.io/api

go.mongodb.org/mongo-driver

go.opentelemetry.io/otel

Cons

1. Have to maintain a domain and static website → Trivial

2. Extra infra can go down:

- Simple and low risk.
- Can be mitigated by go proxy and vendoring.
- Github can and also does go down.

And if it does you can switch your repo root to another forge!

3. DNS issues and domains expiring:

- Not as problematic as changing user names...
- Can recover and re-publish the code from VCS.
- You are probably running an email server with that domain.

If it's all abandoned maybe your code should be abandoned too...

Con: Module In A Subdirectory

`go.0xjac.com/module/src` → `github.com/0xjac/module/src/go.mod`

`go.0xjac.com/module` → `github.com/0xjac/module/src/go.mod` ✨

Real example (source: github.com/golang/go#34055)

`nhooyr.io/websocket` → `https://github.com/nhooyr/websocket/mod`

Fixed in go 1.25: `<meta name="go-import" content="root-path vcs repo-url [subdirectory]">`

`subdirectory`, if present, is the slash-separated subdirectory of the repository that the `root-path` corresponds to, overriding the default of the repository's root directory.

```
<meta name="go-import"
      content="nhooyr.io/websocket git https://github.com/nhooyr/websocket mod">
```

Pro: No forge supports this right now (as far as I can tell).

Question: How does go-source behaves with a subdirectory?

Pros

1. Differentiates your module from `github.com/...`
2. Looks more professional and improves your branding!
3. Full control and more decentralized
Allows you to migrate backend without renaming your module.
4. Use some (exotic) forge/VCS which doesn't provide the go-import meta tag.
5. Full control over go-source.
6. Easily **split your public from private modules** → No one is talking about this!

Separate Public From Private Modules

A company with some but not all open source modules:

Messy `GOPRIVATE=github.com/corp/mod1,github.com/corp/mod3,...`

What if you import a private module in a public one?

Use `depguard` with the same messy configuration...

Solution: go.corp.com and gopriv.corp.org `GOPRIVATE=gopriv.corp.org`

For `depguard` (in public code):

`deny:`

- `pkg: gopriv.copr.org`
- `desc: "Don't import private packages in public packages."`

Problem: Leaking private module names:

1. Make the website private, possible with GH and GL pages!
2. Not a big deal, especially with creative module names.
Can be sneaky by returning 404 with `go-import`.

How To (Not) Do It (and when)!

1. Use dedicated subdomains `go.`, `gopriv.` or dedicated domains: gocorp.com
2. Keep it short: `mygolangmodules.0xjac.com/mymodule`name 😞
3. Use tooling to generate your site: github.com/leighmcculloch/vangen
4. Deploy to your forge's page, netlify, cloudflare, etc.
5. Provide a landing page or redirect for humans to the docs/source
If it's k8s.io (for example)..... you're fucked, it just takes you to a 404 on <https://kubernetes.io>.
Src: lobste.rs
6. Do provide `go-source`.

When:

During a **major version bump** (or if you migrate to another forge)
which requires a **change of import path**.

Copy the code to a new repository and keep the old one for some time (until the go version is not longer supported)

Overview

1. I am not a fan of the term “Vanity” imports, but they look neat and stand out from `github.com/...`
2. Allows full control and extra use cases over your modules.
3. You are **responsible** for your domain, DNS & web pages on top of your code!

Thank you!

Feedback & Questions?

Go Module Reference go.dev/ref/mod
pkg.go.dev source links <https://pkg.go.dev/about#source-links>

VANITY FAIR



WHY YOUR MODULES DESERVE BETTER THAN GITHUB.COM

Because your import path is your first impression.

Jacques Dafflon
0xjac.com